



GLAXIT

INTERNSHIP PROGRAM

Advanced Cyber Security

Developed By: Hasnain Ali

Environment: Kali Linux

LocalStack S3 Bucket Policy Lab

Cloud Security Lab Report — yourname-cloudsec-lab-2026

Environment: Kali Linux | Tools: Docker, LocalStack, awslocal, AWS CLI

1. Overview

This report documents a hands-on lab simulating AWS S3 in a local sandbox using LocalStack running inside Docker on Kali Linux. The lab covers environment setup, provisioning an S3 bucket with the local AWS CLI wrapper (awslocal), uploading an object, and testing the effect of applying and removing a public bucket policy on anonymous read access. Each step below is paired with the terminal output captured during the exercise.

2. Environment Setup

Step 1. Install Docker

Docker Engine and the Docker CLI were installed via apt, since LocalStack runs as a Docker container.

```
(kali@kali)~$ sudo apt install docker.io -y
The following packages were automatically installed and are no longer required:
bluez-hcidump libcarp-clam-perl libdate-calc-xs-perl libtfs-3g99t64 libparse-recdescent-perl libspreadsheet-writexcel-perl libx265-215
firmware-marvell-presteria libcrypt-rc4-perl libfontbox-java libole-storage-lite-perl libpdfbox-java libpdfbox-java libunicode-map-perl network-manager-gnome
libbit-vector-perl libdate-calc-perl libjcode-pm-perl libopenjph0.27 libspreadsheet-parseexcel-perl libvidstab1.1 preview-latex-style
Use 'sudo apt autoremove' to remove them.

Upgrading:
docker-cli docker.io

Summary:
Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 1400
Download size: 27.2 MB
Space needed: 344 kB / 13.5 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 docker-cli amd64 28.5.2+dfsg4-4 [8,049 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 docker.io amd64 28.5.2+dfsg4-4 [19.1 MB]
Fetched 27.2 MB in 29s (926 kB/s)
Preconfiguring packages ...
(Reading database, 835368 files and directories currently installed.)
Preparing to unpack ./docker-cli.28.5.2+dfsg4-4.amd64.deb.
Unpacking docker-cli (28.5.2+dfsg4-4) over (28.5.2+dfsg4-3)...
Preparing to unpack ./docker.io.28.5.2+dfsg4-4.amd64.deb.
Unpacking docker.io (28.5.2+dfsg4-4) over (28.5.2+dfsg4-3)...
Setting up docker-cli (28.5.2+dfsg4-4)...
Setting up docker.io (28.5.2+dfsg4-4)...
Processing triggers for kali-menu (2026.3.2)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes ...
Scanning candidates ...
Scanning linux images ...

Running kernel seems to be up-to-date.

Restarting services ...
Service restarts being deferred:
systemctl restart docker.service

No containers need to be restarted.

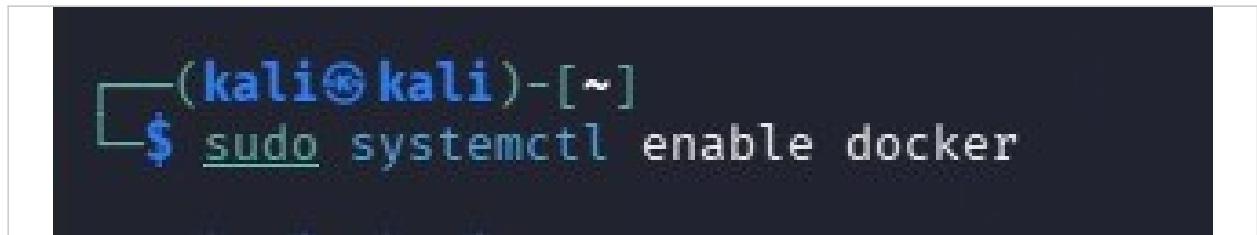
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

Figure 1 — `sudo apt install docker.io -y`

Step 2. Enable the Docker service

The Docker systemd service was enabled so it starts automatically on boot.

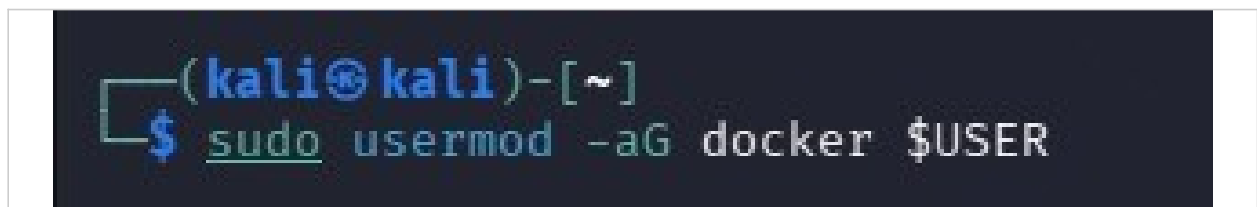
A terminal window with a dark background. The prompt is (kali@kali)-[~]. The command sudo systemctl enable docker is entered and executed. The output is not visible in this snippet.

```
(kali@kali)-[~]  
$ sudo systemctl enable docker
```

Figure 2 — `sudo systemctl enable docker`

Step 3. Add the user to the docker group

The current user was added to the docker group to allow running Docker commands without sudo.

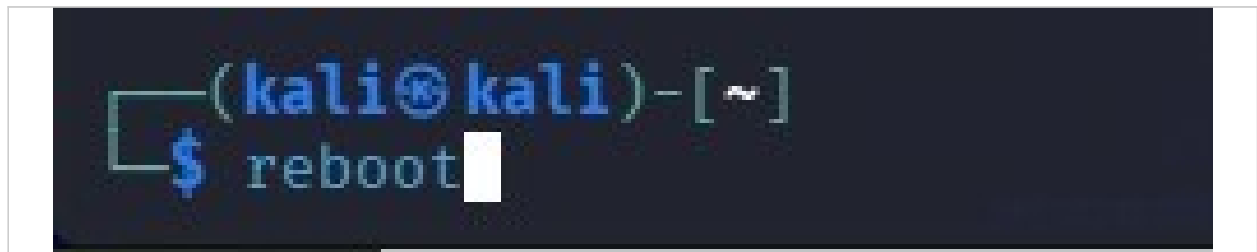
A terminal window with a dark background. The prompt is (kali@kali)-[~]. The command sudo usermod -aG docker \$USER is entered and executed. The output is not visible in this snippet.

```
(kali@kali)-[~]  
$ sudo usermod -aG docker $USER
```

Figure 3 — `sudo usermod -aG docker $USER`

Step 4. Reboot to apply group membership

A reboot was performed so the new group membership took effect for the session.

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The command reboot is entered and executed. The output is not visible in this snippet.

```
(kali@kali)-[~]  
$ reboot
```

Figure 4 — `reboot`

Step 5. Start the Docker service

After reboot, the Docker service was started explicitly.

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The command sudo systemctl start docker is entered and executed. The output is not visible in this snippet.

```
(kali@kali)-[~]  
$ sudo systemctl start docker
```

Figure 5 — `sudo systemctl start docker`

Step 6. Verify Docker is running

`docker ps` was run to confirm the Docker daemon was reachable (no containers running yet).

```
(kali@kali)-[~]
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Figure 6 — docker ps

3. Installing and Configuring LocalStack

Step 7. Install awscli-local

awscli-local (awslocal) was installed via pip, which pulled in boto3, botocore, and the localstack-client package to wrap the AWS CLI against a local endpoint.

```
(kali@kali)-[~]
$ pip install awscli-local --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting awscli-local
  Downloading awscli-local-0.22.2-py3-none-any.whl.metadata (8.0 kB)
Collecting localstack-client (from awscli-local)
  Downloading localstack_client-2.12.tar.gz (11 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: boto3 in /usr/lib/python3/dist-packages (from localstack-client→awscli-local) (1.40.72)
Requirement already satisfied: botocore<1.41.0, ≥1.40.72 in /usr/lib/python3/dist-packages (from boto3→localstack-client→awscli-local) (1.40.72)
Requirement already satisfied: jmespath<2.0.0, ≥0.7.1 in /usr/lib/python3/dist-packages (from boto3→localstack-client→awscli-local) (1.0.1)
Requirement already satisfied: s3transfer<0.15.0, ≥0.14.0 in /usr/lib/python3/dist-packages (from boto3→localstack-client→awscli-local) (0.14.0)
Requirement already satisfied: python-dateutil<3.0.0, ≥2.1 in /usr/lib/python3/dist-packages (from botocore<1.41.0, ≥1.40.72→boto3→localstack-client→awscli-local) (2.9.0)
Requirement already satisfied: urllib3<2.2.0, ≥1.25.4 in /usr/lib/python3/dist-packages (from botocore<1.41.0, ≥1.40.72→boto3→localstack-client→awscli-local) (2.7.0)
Downloading awscli-local-0.22.2-py3-none-any.whl (12 kB)
Building wheels for collected packages: localstack-client
  Building wheel for localstack-client (pyproject.toml) ... done
  Created wheel for localstack-client: filename=localstack_client-2.12-py3-none-any.whl size=10408 sha256=e22852e5a72ebde2017ee2868baf2c50b9c2eb0ec489b3de1c2c0b380b6ccb89
  Stored in directory: /home/kali/.cache/pip/wheels/19/6a/62/b803d1622477cd87b9b9f8a8357c597037741b471109ade12
Successfully built localstack-client
Installing collected packages: localstack-client, awscli-local
Successfully installed awscli-local-0.22.2 localstack-client-2.12
```

Figure 7 — pip install awscli-local --break-system-packages

Step 8. Configure dummy AWS CLI credentials

Since LocalStack does not validate real AWS credentials, placeholder values (test/test) were configured for the AWS CLI, with region us-east-1.

```
(kali@kali)-[~]
$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: test
AWS Secret Access Key [None]: test
Default region name [None]: us-east-1
Default output format [None]: json
```

Figure 8 — aws configure

Step 9. Set the LocalStack auth token

A LocalStack auth token was exported as an environment variable to enable the licensed features used in this lab.

```
(kali@kali)-[~]
$ export LOCALSTACK_AUTH_TOKEN=ls-JedUteKu-0037-hota-FAsE-3822pIdA0a4d
```

Figure 9 — export LOCALSTACK_AUTH_TOKEN=...

Step 10. Start LocalStack

LocalStack was started in detached Docker mode. The CLI activated the cached license and pulled the LocalStack container image before starting it.

```
(kali@kali) (~)
$ localstack start -d
Warning: The 'localstack' CLI is deprecated and will no longer receive updates.
Please use the new 'lstk' CLI: https://docs.localstack.cloud/aws/developer-tools/running-localstack/lstk/migration/

LocalStack
- LocalStack CLI: 2026.8.1
- Profile: default
- App: https://www.localstack.cloud

[21:21:40] starting LocalStack in Docker mode *
2026-09-23T21:21:46.791 INFO [ MainThread] l.p.c.b.licensingv2 : Successfully activated cached license b70dc783-ccc5-4aad-9d0e-82c78510fbd4:trial from /home/kali/.cache/localstack-cli/license.json 🟢
[21:21:47] preparing environment
[21:21:47] configuring container
[21:21:47] container image not found on host
[21:21:48] download complete
[21:21:48] starting container
[21:21:48] detaching
```

Figure 10 — localstack start -d

Step 11. Check LocalStack service status

localstack status services confirmed that the emulated AWS services, including S3 and IAM, were available.

```
(kali@kali) (~)
$ localstack status services
Warning: The 'localstack' CLI is deprecated and will no longer receive updates.
Please use the new 'lstk' CLI: https://docs.localstack.cloud/aws/developer-tools/running-localstack/lstk/migration/

Service      Status
-----
account      ✓ available
acm           ✓ available
acm-pca      ✓ available
amplify      ✓ available
apigateway   ✓ available
apigatewaymanagementapi ✓ available
apigatewayv2 ✓ available
appconfig    ✓ available
appconfigdata ✓ available
application-autoscaling ✓ available
appsync      ✓ available
athena       ✓ available
autoscaling  ✓ available
backup       ✓ available
batch        ✓ available
bedrock      ✓ available
bedrock-agentcore ✓ available
bedrock-agentcore-control ✓ available
bedrock-runtime ✓ available
ce           ✓ available
cloudcontrol ✓ available
cloudformation ✓ available
cloudfront   ✓ available
cloudfront-keyvaluestore ✓ available
cloudtrail   ✓ available
cloudwatch   ✓ available
codeartifact ✓ available
codebuild    ✓ available
codecommit   ✓ available
codeconnections ✓ available
codedeploy   ✓ available
codepipeline ✓ available
codestar-connections ✓ available
cognito-identity ✓ available
cognito-idp  ✓ available
config       ✓ available
dms          ✓ available
docdb        ✓ available
dsq         ✓ available
dynamodb     ✓ available
dynamodbstreams ✓ available
ec2          ✓ available
ecr          ✓ available
ecs          ✓ available
efs          ✓ available
eks          ✓ available
eks-auth     ✓ available
elasticache  ✓ available
elasticbeanstalk ✓ available
elb          ✓ available
elbv2        ✓ available
emr          ✓ available
emr-serverless ✓ available
es           ✓ available
events       ✓ available
firehose     ✓ available
fis          ✓ available
glacier      ✓ available
```

Figure 11 — localstack status services

Step 12. Sanity-check the API with IAM

As a quick sanity check, awslocal iam list-users was run against the local endpoint and returned an empty user list, confirming the CLI was talking to LocalStack correctly.

```
(kali@kali)-[~]  
$ awslocal iam list-users  
{  
  "Users": []  
}
```

Figure 12 — awslocal iam list-users

4. Provisioning the S3 Bucket

Step 13. Create the S3 bucket

An S3 bucket named yourname-cloudsec-lab-2026 was created and listed to confirm it existed.

```
(kali@kali)-[~]  
$ awslocal s3 mb s3://yourname-cloudsec-lab-2026  
awslocalmake_bucket: yourname-cloudsec-lab-2026  
  
(kali@kali)-[~]  
$ awslocal s3 ls  
2026-09-23 21:47:39 yourname-cloudsec-lab-2026
```

Figure 13 — awslocal s3 mb / awslocal s3 ls

Step 14. Upload a test object

A local test.txt file containing sample non-sensitive data was created and uploaded into the bucket.

```
(kali@kali)-[~]  
$ echo "test data - not sensitive" > test.txt  
  
(kali@kali)-[~]  
$ awslocal s3 cp test.txt s3://yourname-cloudsec-lab-2026/  
upload: ./test.txt to s3://yourname-cloudsec-lab-2026/test.txt
```

Figure 14 — echo test data | awslocal s3 cp test.txt s3://.../

5. Testing the Bucket Policy

Step 15. Test anonymous access before applying a policy

Before any bucket policy was applied, an unauthenticated curl request to the object returned HTTP 200 OK, since LocalStack's default S3 endpoint served the object directly.

```
(kali@kali)-[~]
$ curl -I http://localhost:4566/yourname-cloudsec-lab-2026/test.txt
HTTP/1.1 200 OK
Server: TwistedWeb/26.4.0
Date: Thu, 24 Sep 2026 05:16:53 GMT
Content-Type: text/plain
accept-ranges: bytes
Last-Modified: Thu, 24 Sep 2026 05:13:42 GMT
Content-Length: 26
ETag: "d836d047d915c1976f7072958dc6f9c1"
x-amz-server-side-encryption: AES256
x-amz-request-id: 2bba04e3-c873-4b94-9b0e-c551f847617a
x-amz-id-2: s9LzHYrFp76ZVxRcpX9+5cjAnEH2R0uNkd2BHfIa6UkFVdtjf5mKR3/eTPFvsiP/XV/VLi31234=
x-localstack: 2026.8.4
```

Figure 15 — `curl -I http://localhost:4566/.../test.txt` (before policy)

Step 16. Author a public-read bucket policy

A bucket policy document was written granting `s3:GetObject` to any principal ("Principal": "*") on all objects in the bucket, then validated with `python3 -m json.tool`.

```
(kali@kali)-[~]
$ cat > policy.json << 'EOF'
{
  "Version" : "2012-10-17",
  "Statement" : [
    {
      "Effect" : "Allow",
      "Principal" : "*",
      "Action" : "s3:GetObject",
      "Resource" : "arn:aws:s3:::yourname-cloudsec-lab-2026/*"
    }
  ]
}
EOF

(kali@kali)-[~]
$ python3 -m json.tool policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::yourname-cloudsec-lab-2026/*"
    }
  ]
}
```

Figure 16 — `policy.json` (public `GetObject` statement)

Step 17. Apply the policy and re-test access

The policy was applied with `awslocal s3api put-bucket-policy`, and the same `curl` request confirmed the object was still reachable and returned HTTP 200 OK, now explicitly under the public-read policy.

```
(kali@kali)-[~]
$ awslocal s3api put-bucket-policy --bucket yourname-cloudsec-lab-2026 --policy file://policy.json

(kali@kali)-[~]
$ curl -I http://localhost:4566/yourname-cloudsec-lab-2026/test.txt
HTTP/1.1 200 OK
Server: TwistedWeb/26.4.0
Date: Sat, 26 Sep 2026 05:52:07 GMT
Content-Type: text/plain
accept-ranges: bytes
Last-Modified: Thu, 24 Sep 2026 05:13:42 GMT
Content-Length: 26
ETag: "d836d047d915c1976f7072958dc6f9c1"
x-amz-server-side-encryption: AES256
x-amz-request-id: e9870924-3b6a-4c73-91a5-1c35b3ead101
x-amz-id-2: s9LzHYrFp76ZVxRcpX9+5cjAnEH2R0uNkd2BHfIa6UkFVdtjf5mKR3/eTPFvsiP/XV/VLi31234=
x-localstack: 2026.8.4
```

Figure 17 — put-bucket-policy, then curl -I (after policy)

Step 18. Remove the bucket policy

The bucket policy was deleted with delete-bucket-policy. A follow-up get-bucket-policy call correctly returned a NoSuchBucketPolicy error, confirming the policy had been removed.

```
(kali@kali)-[~]
$ awslocal s3api delete-bucket-policy --bucket yourname-cloudsec-lab-2026

(kali@kali)-[~]
$ awslocal s3api get-bucket-policy --bucket yourname-cloudsec-lab-2026
aws: [ERROR]: An error occurred (NoSuchBucketPolicy) when calling the GetBucketPolicy operation: The bucket policy does not exist
```

Figure 18 — delete-bucket-policy / get-bucket-policy (NoSuchBucketPolicy)

6. Summary

This lab demonstrated the full lifecycle of an S3 bucket policy in a local AWS-emulated environment: standing up Docker and LocalStack, provisioning a bucket and object via awslocal, confirming baseline access, applying an explicit public-read bucket policy, verifying its effect, and then removing it. The exercise reinforces how bucket policies control anonymous object access and how to validate policy changes using simple HTTP checks (curl -I) alongside the AWS CLI.